Slide 1: Sliding Window Sums & Hotspots

Speaker : Md Fuzail

Strategy Overview

Problem
Parallel analysis of 2D heatmap — Sliding Window Sums & Hotspots.

Approach
Data parallelism via Row-Major & Column-Major Traversal.

Memory Efficiency
Chunking arrays spatially to maximize cache locality and minimize misses.

Scheduling
`schedule(static)` mapped for uniformly distributed workloads.

Hashing Loop
Collapsed 2D boundaries into a 1D flat loop for compiler optimization.

Thread Safety
`reduction(+:var)` prevents race conditions on shared counters.

</> heatmap_analysis.c

```
// Initialize matrix A with random values — Ehtesham
#pragma omp parallel for schedule(static)
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < columns; ++j) {
        /* generate & assign my_rand() */
    }
}

// Hash A → B applying work_factor iterations — Md Fu
#pragma omp parallel for schedule(static)
for (size_t k = 0; k < total_cells; ++k) {
    /* compute hash(val) w times */
    B[k] = val;
}

// Max sliding window sums per column — Ehtesham Fara
#pragma omp parallel for schedule(static)
for (int j = 0; j < columns; ++j) {
    /* slide window down column j to find max */
    max_sums[j] = max_sum;
}

// Count hotspots with reduction — Md Fuzail
#pragma omp parallel for reduction(+:total_hotspots)
for (int i = 0; i < rows; ++i) {
    /* check 4 neighbours: up, down, left, right */
    hotspots_per_row[i] = row_count;
}
```

Memory Access

Static Schedule

